

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 6
IMPLEMENTASI DOUBLE LINKED LIST PADA BAHASA
PEMROGRAMAN JAVA



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia nya laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman terhadap materi praktikum mengenai implementasi double linked list menggunakan bahasa pemrograman java.

Dalam praktikum ini dibahas berbagai operasi dasar double linked list, seperti penambahan node di awal, akhir, dan posisi tertentu, penelusuran data secara maju maupun mundur, serta penghapusan node pada posisi tertentu.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa mendatang.

Padang, 14 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	5
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Dasar Teori	6
2.2 Langkah Kerja.....	7
2.2.1 Kode NodeDLL_2511532024	7
2.2.2 Penjelasan Kode NodeDLL_2511532024.....	7
2.2.3 Kode InsertDLL_2511532024	9
2.2.4 Penjelasan Kode InsertDLL_2511532024.....	11
2.2.5 Kode PenelusuranDLL_2511532024	16
2.2.6 Penjelasan Kode PenelusuranDLL_2511532024.....	17
2.2.7 Kode HapusDLL_2511532024	20
2.2.8 Penjelasan Kode HapusDLL_2511532024.....	22
2.2.9 Output Kode Program.....	27
BAB III PENUTUP	29
3.1 Kesimpulan	29
DAFTAR PUSTAKA	30
TUGAS PRAKTIKUM STRUKTUR DATA	31
4.1 Soal Tugas	31
4.2 Kode program.....	31
4.2.1 Class Lagu_2511532024.....	31
4.2.2 Class Musik_251153224.....	31
4.3 Penjelasan.....	35
4.3.1 Class Lagu_2511532024.....	35
4.3.2 Class Musik_2511532024.....	36
4.4 Output Kode Program.....	43

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menyebabkan kebutuhan terhadap pengolahan data menjadi semakin besar dan kompleks. Dalam pembuatan suatu program, data harus dikelola dengan baik agar proses pengolahan dapat berjalan secara cepat, tepat, dan efisien. Untuk mendukung hal tersebut, diperlukan penggunaan struktur data dapat dilakukan secara optimal.

Dalam pemrograman, terdapat berbagai jenis struktur data yang digunakan untuk menyimpan dan mengatur data. Salah satu struktur data yang cukup penting adalah linked list. Struktur data ini banyak digunakan karena mampu mengatasi keterbatasan array, terutama dalam hal fleksibilitas ukuran penyimpanan data. Dengan sifatnya yang dinamis, linked list memungkinkan penambahan dan penghapusan data dilakukan dengan lebih mudah selama program dijalankan.

Salah satu bentuk pengembangan dari linked list adalah double linked list, yaitu struktur data yang memiliki hubungan dua arah antar node. Setiap node tidak hanya terhubung ke node berikutnya, tetapi juga terhubung ke node sebelumnya. Hal tersebut membuat proses penelusuran data menjadi lebih fleksibel karena dapat dilakukan dari depan maupun dari belakang. Selain itu, proses manipulasi data seperti penyisipan dan penghapusan node dapat dilakukan dengan lebih efisien.

Pemahaman mengenai double linked list sangat penting dalam pembelajaran struktur data karena konsep ini menjadi dasar dalam pengembangan berbagai aplikasi dan sistem komputer. Oleh sebab itu, pada praktikum ini dilakukan implementasi operasi-operasi dasar double linked list menggunakan bahasa pemrograman java, seperti proses insertion, traversal, dan deletion. Melalui kegiatan praktikum ini diharapkan mahasiswa dapat memahami cara kerja double linked list serta mampu mengimplementasikan dalam pemrograman.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data double linked list.
2. Memahami cara kerja node serta penggunaan pointer next dan prev pada double linked list.
3. Mengimplementasikan operasi dasar double linked list menggunakan bahasa pemrograman java.
4. Menerapkan proses penambahan node di awal, akhir, dan posisi tertentu pada double linked list.
5. Menerapkan proses penelusuran data secara maju dan mundur pada double linked list.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Mahasiswa dapat memahami penerapan struktur data double linked list dalam pemrograman java.
2. Mahasiswa dapat meningkatkan kemampuan dalam mengelola data menggunakan struktur data dinamis.
3. Mahasiswa dapat memahami proses insertion, traversal, dan deletion pada double linked list.
4. Mahasiswa dapat melatih kemampuan analisis dan penyelesaian masalah dalam pemrograman.
5. Mahasiswa memperoleh pengalaman dalam membuat program berbasis struktur data yang terstruktur dan efisien.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan metode yang digunakan untuk menyimpan, mengatur, dan mengelola data di dalam komputer agar proses pengolahan data dapat dilakukan secara efisien. Dalam pemrograman, pemilihan struktur data sangat penting karena berpengaruh terhadap performa program, terutama pada proses pencarian, penambahan, maupun penghapusan data. Struktur data membantu pengorganisasian data sehingga proses manipulasi data menjadi lebih efektif dan terstruktur.

Salah satu struktur data linear yang sering digunakan adalah linked list. Linked list merupakan kumpulan node yang saling terhubung melalui pointer atau referensi. Setiap node umumnya terdiri dari data dan pointer menuju node lainnya. Berbeda dengan array yang memiliki ukuran tetap dan lokasi memori berurutan, linked list bersifat dinamis sehingga ukuran data dapat bertambah maupun berkurang selama program dijalankan. Struktur data ini memiliki keunggulan dalam fleksibilitas pengelolaan data karena proses penambahan dan penghapusan elemen dapat dilakukan tanpa memindahkan seluruh data yang ada.

Salah satu pengembangan dari linked list adalah double linked list. Pada struktur data ini, setiap node memiliki dua pointer, yaitu pointer menuju node berikutnya (next) dan pointer menuju node sebelumnya (prev). Dengan adanya dua pointer tersebut, penelusuran data dapat dilakukan ke dua arah, yaitu maju dan mundur. Struktur ini memberikan kemudahan dalam proses navigasi data, penambahan node, maupun penghapusan node pada posisi tertentu. Double linked list memiliki fleksibilitas traversal yang lebih baik dibandingkan single linked list, meskipun membutuhkan penggunaan memori lebih besar karena adanya tambahan pointer pada setiap node.

Dalam implementasinya, double linked list memiliki beberapa operasi dasar, yaitu insertion, traversal, dan deletion. Operasi insertion

digunakan untuk menambahkan node baru di awal, akhir, atau posisi tertentu dalam list. Traversal digunakan untuk menelusuri seluruh node baik secara maju maupun mundur menggunakan pointer next dan prev. sementara itu, deletion digunakan untuk menghapus node tertentu dari list dengan memperbarui hubungan antar node agar struktur data tetap terhubung dengan baik. Operasi-operasi tersebut menjadi dasar penting dalam pengelolaan data dinamis pada berbagai aplikasi pemrograman.

2.2 Langkah Kerja

2.2.1 Kode NodeDLL_2511532024

```
1. public class NodeDLL_2511532024 {
2. // mendefinisikan kelas node
3. int data_2024; // data
4. NodeDLL_2511532024 next_2024; // pointer ke next
   node
5. NodeDLL_2511532024 prev_2024; // pointer ke previous
   node
6.
7. // konstruktor
8. public NodeDLL_2511532024(int data_2024) {
9.     this.data_2024 = data_2024;
10.    this.next_2024 = null;
11.    this.prev_2024 = null;
12. }
13. }
```

2.2.2 Penjelasan Kode NodeDLL_2511532024

1. Deklarasi Class

```
public class NodeDLL_2511532024 {
```

Syntax tersebut digunakan untuk mendeklarasikan class bernama NodeDLL_2511532024. Class ini berfungsi sebagai template atau cetakan untuk membuat objek node pada double linked list.

2. Deklarasi Variabel Data dan Pointer

```
int data_2024;
NodeDLL_2511532024 next_2024;
NodeDLL_2511532024 prev_2024;
```

Syntax `int data_2024` digunakan untuk menyimpan nilai atau data pada node. Selanjutnya, syntax `NodeDLL_2511532024 next_2024` digunakan sebagai pointer atau referensi yang menunjuk ke node berikutnya, sedangkan syntax `NodeDLL_2511532024 prev_2024` digunakan sebagai pointer atau referensi yang menunjuk ke node sebelumnya. Karena menggunakan Doubly Linked List, setiap node dapat diakses ke dua arah, yaitu maju dan mundur.

3. Konstruktor Node

```
public NodeDLL_2511532024(int data_2024) {
```

Syntax tersebut merupakan konstruktor yang digunakan untuk membuat objek node baru sekaligus menerima nilai data yang akan disimpan ke dalam node. Konstruktor akan otomatis dijalankan ketika objek node dibuat.

4. Inisialisasi Data dan Pointer

```
this.data_2024 = data_2024;  
this.next_2024 = null;  
this.prev_2024 = null;
```

Syntax `this.data_2024 = data_2024;` digunakan untuk mengisi atribut `data_2024` dengan nilai yang dikirim melalui parameter konstruktor. Selanjutnya, syntax `this.next_2024 = null;` digunakan untuk memberikan nilai awal `null` pada pointer `next_2024`, yang menandakan bahwa node belum terhubung dengan node berikutnya. Kemudian, syntax `this.prev_2024 = null;` digunakan untuk memberikan nilai awal `null` pada pointer `prev_2024`, yang berarti node juga belum terhubung dengan node sebelumnya. Pemberian nilai `null` pada kedua pointer dilakukan karena node yang baru dibuat masih berdiri sendiri dan belum terhubung dengan node lain di dalam Doubly Linked List.

2.2.3 Kode InsertDLL_2511532024

```
1. public class InsertDLL_2511532024 {
2.     // menambahkan node diawal DLL
3.     static NodeDLL_2511532024
insertBegin_2024(NodeDLL_2511532024 head_2024, int
data_2024) {
4.         // buat node baru
5.         NodeDLL_2511532024 new_node_2024 = new
NodeDLL_2511532024(data_2024);
6.         // jadikan pointer nextnya head
7.         new_node_2024.next_2024 = head_2024;
8.         // jadikan pointer prev head ke new_node
9.         if (head_2024 != null) {
10.            head_2024.prev_2024 = new_node_2024;
11.        }
12.        return new_node_2024;
13.    }
14.    //fungsi menambahkan node diakhir
15.    public static NodeDLL_2511532024
insertEnd_2024(NodeDLL_2511532024 head_2024, int
newData_2024) {
16.        // buat node baru
17.        NodeDLL_2511532024 newNode_2024 = new
NodeDLL_2511532024(newData_2024);
18.        // jika dll null dijadikan head
19.        if (head_2024 == null) {
20.            head_2024 = newNode_2024;
21.        } else {
22.            NodeDLL_2511532024 curr_2024 =
head_2024;
23.            while (curr_2024.next_2024 != null) {
24.                curr_2024 = curr_2024.next_2024;
25.            }
26.            curr_2024.next_2024 = newNode_2024;
27.            newNode_2024.prev_2024 = curr_2024;
28.        }
29.        return head_2024;
30.    }
31.    // fungsi menambahkan node di posisi tertentu
32.    public static NodeDLL_2511532024
insertAtPosition_2024(NodeDLL_2511532024 head_2024,
int pos_2024, int new_data_2024) {
33.        // buat node baru
34.        NodeDLL_2511532024 new_node_2024 = new
NodeDLL_2511532024(new_data_2024);
35.        if (pos_2024 == 1) {
36.            new_node_2024.next_2024 = head_2024;
37.            if (head_2024 != null) {
38.                head_2024.prev_2024 =
new_node_2024;
39.            }
40.            head_2024 = new_node_2024;
41.            return head_2024;
42.        }
```

```

43.     NodeDLL_2511532024 curr_2024 = head_2024;
44.
45.     for (int i_2024 = 1; i_2024 < pos_2024 - 1 &&
curr_2024 != null; ++i_2024) {
46.         curr_2024 = curr_2024.next_2024;
47.     }
48.     if (curr_2024 == null) {
49.         System.out.println("Posisi tidak
ada");
50.         return head_2024;
51.     }
52.     new_node_2024.prev_2024 = curr_2024;
53.     new_node_2024.next_2024 =
curr_2024.next_2024;
54.     curr_2024.next_2024 = new_node_2024;
55.     if (new_node_2024.next_2024 != null) {
56.         new_node_2024.next_2024.prev_2024 = new_node_2024;
57.     }
58.     return head_2024;
59. }
60.
61. public static void printList_2024(NodeDLL_2511532024
head_2024) {
62.     NodeDLL_2511532024 curr_2024 = head_2024;
63.     while (curr_2024 != null) {
64.         System.out.print(curr_2024.data_2024 +
" <-> ");
65.         curr_2024 = curr_2024.next_2024;
66.     }
67.     System.out.println();
68. }
69.
70. public static void main(String[] args) {
71.     // membuat dll 2 <-> 3 <-> 5
72.     NodeDLL_2511532024 head_2024 = new
NodeDLL_2511532024(2);
73.     head_2024.next_2024 = new
NodeDLL_2511532024(3);
74.     head_2024.next_2024.prev_2024 = head_2024;
75.     head_2024.next_2024.next_2024 = new
NodeDLL_2511532024(5);
76.     head_2024.next_2024.next_2024.prev_2024 =
head_2024.next_2024;
77.     // cetak DLL awal
78.     System.out.print("DLL Awal: ");
79.     printList_2024(head_2024);
80.     // tambah 1 diawal
81.     head_2024 = insertBegin_2024(head_2024, 1);
82.     System.out.print("Simpul 1 ditambah di awal:
");
83.     printList_2024(head_2024);
84.     // tambah 6 diakhir
85.     System.out.print("Simpul 6 di tambah di akhir:
");
86.     printList_2024(head_2024);
87.     int data_2024 = 6;

```

```

88.     head_2024 = insertEnd_2024(head_2024,
    data_2024);
89.     printList_2024(head_2024);
90.     // menambahkan node 4 diposisi 4
91.     System.out.print("tambah node 4 di posisi 4:
    ");
92.     int data2_2024 = 4;
93.     int pos_2024 = 4;
94.     head_2024 = insertAtPosition_2024(head_2024,
    pos_2024, data2_2024);
95.     printList_2024(head_2024);
96. }
97. }

```

2.2.4 Penjelasan Kode InsertDLL_2511532024

1. Fungsi insertBegin_2024()

```

static NodeDLL_2511532024
insertBegin_2024(NodeDLL_2511532024 head_2024, int
data_2024)

```

Fungsi insertBegin_2024() digunakan untuk menambahkan node baru di bagian awal Double Linked List. Fungsi menerima parameter head_2024 sebagai node pertama pada list dan data_2024 sebagai data yang akan dimasukkan ke node baru.

```

NodeDLL_2511532024    new_node_2024    =    new
NodeDLL_2511532024(data_2024);

```

Syntax tersebut digunakan untuk membuat node baru dengan data yang diterima dari parameter. Node baru akan disimpan pada variabel new_node_2024.

```

new_node_2024.next_2024 = head_2024;

```

Syntax tersebut digunakan untuk menghubungkan pointer next_2024 milik node baru ke node head lama. Dengan demikian, node baru akan berada di depan list.

```

if (head_2024 != null) {
    head_2024.prev_2024 = new_node_2024;
}

```

Bagian ini digunakan untuk mengatur pointer prev_2024 pada head lama agar menunjuk kembali ke node baru.

Kondisi if digunakan untuk memastikan bahwa list tidak kosong sebelum pointer diubah.

```
return new_node_2024;
```

Syntax tersebut digunakan untuk mengembalikan node baru sebagai head terbaru pada Double Linked List.

2. Fungsi insertEnd_2024()

```
public static NodeDLL_2511532024  
insertEnd_2024(NodeDLL_2511532024 head_2024, int  
newData_2024)
```

Fungsi ini digunakan untuk menambahkan node baru di bagian akhir Double Linked List.

```
NodeDLL_2511532024 newNode_2024 = new  
NodeDLL_2511532024(newData_2024);
```

Syntax tersebut digunakan untuk membuat node baru yang akan ditambahkan ke akhir list.

```
if (head_2024 == null) {  
    head_2024 = newNode_2024;  
}
```

Kondisi tersebut digunakan untuk memeriksa apakah Double Linked List masih kosong. Jika kosong, maka node baru langsung dijadikan sebagai head.

```
NodeDLL_2511532024 curr_2024 = head_2024;  
while (curr_2024.next_2024 != null) {  
    curr_2024 = curr_2024.next_2024;  
}
```

Bagian ini digunakan untuk melakukan traversal menuju node terakhir. Variabel curr_2024 bergerak dari head sampai pointer next_2024 bernilai null, yang menandakan posisi node terakhir telah ditemukan.

```
curr_2024.next_2024 = newNode_2024;  
newNode_2024.prev_2024 = curr_2024;
```

Syntax tersebut digunakan untuk menghubungkan node terakhir dengan node baru. Pointer next_2024 pada node terakhir diarahkan ke node baru, sedangkan pointer prev_2024 pada node baru diarahkan kembali ke node terakhir.

```
return head_2024;
```

Syntax tersebut digunakan untuk mengembalikan head dari Double Linked List setelah proses penambahan node selesai dilakukan.

3. Fungsi insertAtPosition_2024()

```
public static NodeDLL_2511532024  
insertAtPosition_2024(NodeDLL_2511532024 head_2024,  
int pos_2024, int new_data_2024)
```

Fungsi ini digunakan untuk menambahkan node pada posisi tertentu di dalam Double Linked List.

```
NodeDLL_2511532024 new_node_2024 = new  
NodeDLL_2511532024(new_data_2024);
```

Syntax tersebut digunakan untuk membuat node baru yang akan disisipkan ke posisi tertentu.

```
if (pos_2024 == 1)
```

Kondisi ini digunakan untuk memeriksa apakah node akan ditambahkan pada posisi pertama. Jika posisi bernilai 1, maka proses insertion dilakukan seperti penambahan di awal list.

```
NodeDLL_2511532024 curr_2024 = head_2024;
```

Variabel curr_2024 digunakan sebagai pointer bantu untuk melakukan traversal menuju posisi yang diinginkan.

```
for (int i_2024 = 1; i_2024 < pos_2024 - 1 && curr_2024 !=  
null; ++i_2024) {  
    curr_2024 = curr_2024.next_2024;  
}
```

Perulangan tersebut digunakan untuk menelusuri node sampai posisi sebelum lokasi penyisipan node baru.

```
if (curr_2024 == null) {  
    System.out.println("Posisi tidak ada");  
    return head_2024;  
}
```

Bagian ini digunakan untuk memeriksa apakah posisi yang diminta tersedia. Jika posisi tidak ditemukan, program akan menampilkan pesan kesalahan dan mengembalikan list tanpa perubahan.

```
new_node_2024.prev_2024 = curr_2024;  
new_node_2024.next_2024 = curr_2024.next_2024;  
curr_2024.next_2024 = new_node_2024;
```

Syntax tersebut digunakan untuk menyisipkan node baru ke dalam list. Pointer prev_2024 dan next_2024 diatur agar node baru terhubung dengan node sebelum dan sesudahnya.

```
if (new_node_2024.next_2024 != null) {  
    new_node_2024.next_2024.prev_2024 =  
    new_node_2024;  
}
```

Bagian ini digunakan untuk memperbarui pointer prev_2024 pada node setelah node baru agar hubungan antar node tetap benar.

4. Fungsi printList_2024()

```
public static void printList_2024(NodeDLL_2511532024  
head_2024)
```

Fungsi printList_2024() digunakan untuk menampilkan seluruh isi Doubly Linked List.

```
while (curr_2024 != null) {  
    System.out.print(curr_2024.data_2024 + " <-> ");  
    curr_2024 = curr_2024.next_2024;
```

```
}
```

Perulangan tersebut digunakan untuk melakukan traversal dari head hingga node terakhir sambil menampilkan data pada setiap node.

5. Fungsi main()

```
public static void main(String[] args)
```

Fungsi main() merupakan bagian utama program yang digunakan untuk menjalankan seluruh operasi pada Double Linked List.

```
NodeDLL_2511532024 head_2024 = new  
NodeDLL_2511532024(2);  
head_2024.next_2024 = new NodeDLL_2511532024(3);  
head_2024.next_2024.prev_2024 = head_2024;  
head_2024.next_2024.next_2024 = new  
NodeDLL_2511532024(5);  
head_2024.next_2024.next_2024.prev_2024 =  
head_2024.next_2024;
```

Syntax tersebut digunakan untuk membuat Double Linked List awal dengan data 2 <-> 3 <-> 5. Node pertama dijadikan sebagai head, kemudian node kedua dan ketiga ditambahkan serta dihubungkan menggunakan pointer next_2024 dan prev_2024 agar hubungan antar node terbentuk ke dua arah.

```
System.out.print("DLL Awal: ");  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menampilkan isi awal Double Linked List sebelum dilakukan proses penambahan node.

```
head_2024 = insertBegin_2024(head_2024, 1);  
System.out.print("Simpul 1 ditambah di awal: ");  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menambahkan node bernilai 1 di bagian awal Double Linked List, kemudian menampilkan hasil list setelah penambahan dilakukan.

```
int data_2024 = 6;
head_2024 = insertEnd_2024(head_2024, data_2024);
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menambahkan node bernilai 6 di bagian akhir Double Linked List, lalu menampilkan isi list setelah node berhasil ditambahkan.

```
int data2_2024 = 4;
int pos_2024 = 4;
head_2024 = insertAtPosition_2024(head_2024, pos_2024,
data2_2024);
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menambahkan node bernilai 4 pada posisi ke-4 di dalam Double Linked List, kemudian menampilkan hasil akhir list setelah proses insertion selesai dilakukan.

2.2.5 Kode PenelusuranDLL_2511532024

```
1. public class PenelusuranDLL_2511532024 {
2. // fungsi penelusuran maju
3. static void forwardTraversal_2024(NodeDLL_2511532024
head_2024) {
4. // memulai penelusuran dari head
5. NodeDLL_2511532024 curr_2024 = head_2024;
6. // lanjutkan sampai akhir
7. while (curr_2024 != null) {
8. // print data
9. System.out.print(curr_2024.data_2024 +
" <-> ");
10. // pindah ke node berikutnya
11. curr_2024 = curr_2024.next_2024;
12. }
13. // print spasi
14. System.out.println();
15. }
16. // fungsi penelusuran mundur
17. static void
backwardTraversal_2024(NodeDLL_2511532024 tail_2024)
{
```



```

18.      // mulai dari akhir
19.      NodeDLL_2511532024 curr_2024 = tail_2024;
20.      // lanjut sampai head
21.      while (curr_2024 != null) {
22.          //cetak data
23.          System.out.print(curr_2024.data_2024 +
" <-> ");
24.          // pindah ke node sebelumnya
25.          curr_2024 = curr_2024.prev_2024;
26.      }
27.      // cetak spasi
28.      System.out.println();
29. }
30. public static void main(String[] args) {
31.     // cetak DLL
32.     NodeDLL_2511532024 head_2024 = new
NodeDLL_2511532024(1);
33.     NodeDLL_2511532024 second_2024 = new
NodeDLL_2511532024(2);
34.     NodeDLL_2511532024 third_2024 = new
NodeDLL_2511532024(3);
35.
36.     head_2024.next_2024 = second_2024;
37.     second_2024.prev_2024 = head_2024;
38.     second_2024.next_2024 = third_2024;
39.     third_2024.prev_2024 = second_2024;
40.
41.     System.out.println("Penelusuran maju: ");
42.     forwardTraversal_2024(head_2024);
43.
44.     System.out.println("Penelusuran mundur: ");
45.     backwardTraversal_2024(third_2024);
46. }
47. }

```

2.2.6 Penjelasan Kode PenelusuranDLL_2511532024

1. Fungsi forwardTraversal_2024()

```
static void forwardTraversal_2024(NodeDLL_2511532024
head_2024)
```

Fungsi forwardTraversal_2024() digunakan untuk melakukan penelusuran maju pada Double Linked List. Parameter head_2024 digunakan sebagai titik awal traversal karena proses dimulai dari node pertama.

```
NodeDLL_2511532024 curr_2024 = head_2024;
while (curr_2024 != null) {
    System.out.print(curr_2024.data_2024 + " <-> ");
    curr_2024 = curr_2024.next_2024;
```

```
}  
System.out.println();
```

Syntax tersebut digunakan untuk melakukan traversal maju pada Double Linked List. Variabel `curr_2024` digunakan sebagai pointer bantu yang dimulai dari `head_2024`. Perulangan `while` akan terus berjalan selama pointer belum bernilai `null`. Pada setiap perulangan, data node akan ditampilkan menggunakan `System.out.print()`, kemudian pointer berpindah ke node berikutnya melalui `curr_2024.next_2024`. Setelah seluruh node selesai ditelusuri, `System.out.println()` digunakan untuk mencetak baris baru.

2. Fungsi `backwardTraversal_2024()`

```
static void  
backwardTraversal_2024(NodeDLL_2511532024 tail_2024)
```

Fungsi `backwardTraversal_2024()` digunakan untuk melakukan penelusuran mundur pada Double Linked List. Parameter `tail_2024` digunakan sebagai titik awal traversal karena proses dimulai dari node terakhir.

```
NodeDLL_2511532024 curr_2024 = tail_2024;  
while (curr_2024 != null) {  
    System.out.print(curr_2024.data_2024 + " <-> ");  
    curr_2024 = curr_2024.prev_2024;  
}  
System.out.println();
```

Syntax tersebut digunakan untuk melakukan traversal mundur pada Double Linked List. Variabel `curr_2024` digunakan sebagai pointer bantu yang dimulai dari node terakhir. Perulangan `while` akan berjalan selama pointer belum bernilai `null`. Pada setiap perulangan, data node akan ditampilkan, kemudian pointer berpindah ke node sebelumnya menggunakan `curr_2024.prev_2024`. Setelah

traversal selesai dilakukan, program mencetak baris baru menggunakan `System.out.println()`.

3. Fungsi main()

```
NodeDLL_2511532024    head_2024    =    new
NodeDLL_2511532024(1);
NodeDLL_2511532024    second_2024    =    new
NodeDLL_2511532024(2);
NodeDLL_2511532024    third_2024    =    new
NodeDLL_2511532024(3);
```

Syntax tersebut digunakan untuk membuat tiga buah node dengan data 1, 2, dan 3.

```
head_2024.next_2024 = second_2024;
second_2024.prev_2024 = head_2024;
second_2024.next_2024 = third_2024;
third_2024.prev_2024 = second_2024;
```

Syntax tersebut digunakan untuk menghubungkan seluruh node sehingga membentuk struktur Doubly Linked List dengan urutan 1 <-> 2 <-> 3. Pointer `next_2024` digunakan untuk menghubungkan node ke arah depan, sedangkan pointer `prev_2024` digunakan untuk menghubungkan node ke arah belakang.

```
System.out.println("Penelusuran maju: ");
forwardTraversal_2024(head_2024);
```

Syntax tersebut digunakan untuk menampilkan teks penanda dan menjalankan proses traversal maju mulai dari node head.

```
System.out.println("Penelusuran mundur: ");
backwardTraversal_2024(third_2024);
```

Syntax tersebut digunakan untuk menampilkan teks penanda dan menjalankan proses traversal mundur mulai dari node terakhir.

2.2.7 Kode HapusDLL_2511532024

```
1. public class HapusDLL_2511532024 {
2.     // fungsi menghapus node awal
3.     public static NodeDLL_2511532024
4.     delHead_2024(NodeDLL_2511532024 head_2024) {
5.         if (head_2024 == null) {
6.             return null;
7.         }
8.         NodeDLL_2511532024 temp_2024 = head_2024;
9.         head_2024 = head_2024.next_2024;
10.        if (head_2024 != null) {
11.            head_2024.prev_2024 = null;
12.        }
13.        return head_2024;
14.    }
15.    //fungsi menghapus di akhir
16.    public static NodeDLL_2511532024
17.    dellast_2024(NodeDLL_2511532024 head_2024) {
18.        if (head_2024 == null) {
19.            return null;
20.        }
21.        if (head_2024.next_2024 == null) {
22.            return null;
23.        }
24.        NodeDLL_2511532024 curr_2024 = head_2024;
25.        while (curr_2024.next_2024 != null) {
26.            curr_2024 = curr_2024.next_2024;
27.        }
28.        // update pointer previous node
29.        if (curr_2024.prev_2024 != null) {
30.            curr_2024.prev_2024.next_2024 = null;
31.        }
32.        return head_2024;
33.    }
34.    // fungsi menghapus node posisi tertentu
35.    public static NodeDLL_2511532024
36.    delPos_2024(NodeDLL_2511532024 head_2024, int
37.    pos_2024) {
38.        // jika DLL kosong
39.        if (head_2024 == null) {
40.            return head_2024;
41.        }
42.        NodeDLL_2511532024 curr_2024 = head_2024;
43.        // telusuri sampai ke node yang akan dihapus
44.        for (int i_2024 = 1; curr_2024 != null &&
45.        i_2024 < pos_2024; ++i_2024) {
46.            curr_2024 = curr_2024.next_2024;
47.        }
48.        // jika posisi tidak ditemukan
49.        if (curr_2024 == null) {
50.            return head_2024;
51.        }
52.        // update pointer
53.        if (curr_2024.prev_2024 != null) {
54.            curr_2024.prev_2024.next_2024 =
55.            curr_2024.next_2024;
56.        }
57.        return head_2024;
58.    }
59.}
```

```

50.     }
51.     if (curr_2024.next_2024 != null) {
52.         curr_2024.next_2024.prev_2024 =
curr_2024.prev_2024;
53.     }
54.     // jika yang dihapus head
55.     if (head_2024 == curr_2024) {
56.         head_2024 = curr_2024.next_2024;
57.     }
58.     return head_2024;
59. }
60. // fungsi mencetak DLL
61. public static void printList_2024(NodeDLL_2511532024
head_2024) {
62.     NodeDLL_2511532024 curr_2024 = head_2024;
63.     while (curr_2024 != null) {
64.         System.out.print(curr_2024.data_2024 +
" <-> ");
65.         curr_2024 = curr_2024.next_2024;
66.     }
67.     System.out.println();
68. }
69. public static void main(String[] args) {
70.     // buat sebuah DLL
71.     NodeDLL_2511532024 head_2024 = new
NodeDLL_2511532024(1);
72.     head_2024.next_2024 = new
NodeDLL_2511532024(2);
73.     head_2024.next_2024.prev_2024 = head_2024;
74.     head_2024.next_2024.next_2024 = new
NodeDLL_2511532024(3);
75.     head_2024.next_2024.next_2024.prev_2024 =
head_2024.next_2024;
76.     head_2024.next_2024.next_2024.next_2024 = new
NodeDLL_2511532024(4);
77.     head_2024.next_2024.next_2024.next_2024.prev_2024 =
head_2024.next_2024.next_2024;
78.     head_2024.next_2024.next_2024.next_2024.next_2024 =
new NodeDLL_2511532024(5);
79.     head_2024.next_2024.next_2024.next_2024.next_2024.pr
ev_2024 =
head_2024.next_2024.next_2024.next_2024;
80.
81.     System.out.print("DLL Awal: ");
82.     printList_2024(head_2024);
83.
84.     System.out.print("Setelah head dihapus: ");
85.     head_2024 = delHead_2024(head_2024);
86.     printList_2024(head_2024);
87.
88.     System.out.print("Setelah node terakhir
dihapus: ");
89.     head_2024 = dellast_2024(head_2024);
90.     printList_2024(head_2024);

```

```

91.
92.     System.out.print("menghapus node ke 2: ");
93.     head_2024 = delPos_2024(head_2024, 2);
94.
95.     printList_2024(head_2024);
96. }
97. }

```

2.2.8 Penjelasan Kode HapusDLL_2511532024

1. Fungsi delHead_2024()

```

public          static          NodeDLL_2511532024
delHead_2024(NodeDLL_2511532024 head_2024)

```

Fungsi delHead_2024() digunakan untuk menghapus node pertama atau head pada Double Linked List.

```

if (head_2024 == null) {
    return null;
}

```

Syntax tersebut digunakan untuk memeriksa apakah Double Linked List kosong. Jika head bernilai null, maka tidak ada node yang dapat dihapus sehingga fungsi langsung mengembalikan null.

```

NodeDLL_2511532024 temp_2024 = head_2024;
head_2024 = head_2024.next_2024;

```

Syntax tersebut digunakan untuk menyimpan head lama ke variabel temp_2024, kemudian memindahkan head ke node berikutnya sehingga node pertama dianggap telah dihapus dari list.

```

if (head_2024 != null) {
    head_2024.prev_2024 = null;
}

```

Bagian ini digunakan untuk menghapus hubungan pointer prev_2024 pada head baru agar tidak lagi menunjuk ke node lama yang telah dihapus.

```

return head_2024;

```

Syntax tersebut digunakan untuk mengembalikan head terbaru setelah proses penghapusan selesai dilakukan.

2. Fungsi delLast_2024()

```
public static NodeDLL_2511532024  
delLast_2024(NodeDLL_2511532024 head_2024)
```

Fungsi delLast_2024() digunakan untuk menghapus node terakhir pada Double Linked List.

```
if (head_2024 == null) {  
    return null;  
}  
if (head_2024.next_2024 == null) {  
    return null;  
}
```

Syntax tersebut digunakan untuk memeriksa kondisi Double Linked List. Jika list kosong atau hanya memiliki satu node, maka fungsi langsung mengembalikan null karena tidak ada node lain yang tersisa setelah penghapusan dilakukan.

```
NodeDLL_2511532024 curr_2024 = head_2024;  
while (curr_2024.next_2024 != null) {  
    curr_2024 = curr_2024.next_2024;  
}
```

Syntax tersebut digunakan untuk melakukan traversal menuju node terakhir. Variabel curr_2024 bergerak dari head ke node berikutnya sampai pointer next_2024 bernilai null.

```
if (curr_2024.prev_2024 != null) {  
    curr_2024.prev_2024.next_2024 = null;  
}
```

Bagian ini digunakan untuk memutus hubungan node terakhir dengan node sebelumnya. Pointer next_2024 milik

node sebelum node terakhir diubah menjadi null sehingga node terakhir tidak lagi terhubung dengan list.

```
return head_2024;
```

Syntax tersebut digunakan untuk mengembalikan head setelah proses penghapusan node terakhir selesai dilakukan.

3. Fungsi delPos_2024()

```
public          static          NodeDLL_2511532024  
delPos_2024(NodeDLL_2511532024    head_2024,    int  
pos_2024)
```

Fungsi delPos_2024() digunakan untuk menghapus node pada posisi tertentu di dalam Double Linked List.

```
if (head_2024 == null) {  
    return head_2024;  
}
```

Syntax tersebut digunakan untuk memeriksa apakah list kosong. Jika kosong, maka fungsi langsung mengembalikan head tanpa perubahan.

```
NodeDLL_2511532024 curr_2024 = head_2024;  
for (int i_2024 = 1; curr_2024 != null && i_2024 <  
pos_2024; ++i_2024) {  
    curr_2024 = curr_2024.next_2024;  
}
```

Syntax tersebut digunakan untuk melakukan traversal menuju node yang akan dihapus. Variabel curr_2024 bergerak dari head menuju posisi yang ditentukan menggunakan perulangan for.

```
if (curr_2024 == null) {  
    return head_2024;  
}
```


Bagian ini digunakan untuk memeriksa apakah posisi yang diminta tersedia di dalam list. Jika posisi tidak ditemukan, maka list dikembalikan tanpa perubahan.

```
if (curr_2024.prev_2024 != null) {  
    curr_2024.prev_2024.next_2024 =  
curr_2024.next_2024;  
}  
if (curr_2024.next_2024 != null) {  
    curr_2024.next_2024.prev_2024 =  
curr_2024.prev_2024;  
}
```

Syntax tersebut digunakan untuk memperbaiki hubungan antar node setelah node pada posisi tertentu dihapus. Pointer next_2024 milik node sebelumnya diarahkan ke node setelahnya, sedangkan pointer prev_2024 milik node setelahnya diarahkan ke node sebelumnya.

```
if (head_2024 == curr_2024) {  
    head_2024 = curr_2024.next_2024;  
}
```

Bagian ini digunakan untuk memeriksa apakah node yang dihapus merupakan head. Jika benar, maka head dipindahkan ke node berikutnya.

```
return head_2024;
```

Syntax tersebut digunakan untuk mengembalikan head setelah proses penghapusan selesai dilakukan.

4. Fungsi printList_2024()

```
public static void printList_2024(NodeDLL_2511532024  
head_2024)
```

Fungsi printList_2024() digunakan untuk menampilkan isi Double Linked List.

```

NodeDLL_2511532024 curr_2024 = head_2024;
while (curr_2024 != null) {
    System.out.print(curr_2024.data_2024 + " <-> ");
    curr_2024 = curr_2024.next_2024;
}
System.out.println();

```

Syntax tersebut digunakan untuk melakukan traversal dari head hingga node terakhir sambil menampilkan data pada setiap node. Setelah traversal selesai, program mencetak baris baru menggunakan System.out.println().

5. Fungsi main()

```

NodeDLL_2511532024 head_2024 = new
NodeDLL_2511532024(1);
head_2024.next_2024 = new NodeDLL_2511532024(2);
head_2024.next_2024.prev_2024 = head_2024;
head_2024.next_2024.next_2024 = new
NodeDLL_2511532024(3);
head_2024.next_2024.next_2024.prev_2024 =
head_2024.next_2024;
head_2024.next_2024.next_2024.next_2024 = new
NodeDLL_2511532024(4);
head_2024.next_2024.next_2024.next_2024.prev_2024 =
head_2024.next_2024.next_2024;
head_2024.next_2024.next_2024.next_2024.next_2024 =
new NodeDLL_2511532024(5);
head_2024.next_2024.next_2024.next_2024.next_2024.prev
_2024 = head_2024.next_2024.next_2024.next_2024;

```

Syntax tersebut digunakan untuk membuat dan menghubungkan node sehingga membentuk Doubly Linked List dengan urutan 1 <-> 2 <-> 3 <-> 4 <-> 5. Pointer

next_2024 digunakan untuk hubungan ke depan, sedangkan pointer prev_2024 digunakan untuk hubungan ke belakang.

```
System.out.print("DLL Awal: ");  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menampilkan isi awal Double Linked List sebelum dilakukan proses penghapusan node.

```
head_2024 = delHead_2024(head_2024);  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menghapus node pertama pada list kemudian menampilkan hasil list setelah proses deletion dilakukan.

```
head_2024 = delLast_2024(head_2024);  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menghapus node terakhir pada list kemudian menampilkan hasil list terbaru.

```
head_2024 = delPos_2024(head_2024, 2);  
printList_2024(head_2024);
```

Syntax tersebut digunakan untuk menghapus node pada posisi ke-2 kemudian menampilkan isi list setelah proses penghapusan selesai dilakukan.

2.2.9 Output Kode Program

1. Output Kode Program InsertDLL_2511532024

```
DLL Awal: 2 <-> 3 <-> 5 <->  
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->  
Simpul 6 di tambah di akhir: 1 <-> 2 <-> 3 <-> 5 <->  
1 <-> 2 <-> 3 <-> 5 <-> 6 <->  
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2. Output Kode Program PenelusuranDLL_2511532024

```
Penelusuran maju:  
1 <-> 2 <-> 3 <->  
Penelusuran mundur:  
3 <-> 2 <-> 1 <->
```

3. Output Kode Program HapusDLL_2511532024

```
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->  
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->  
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->  
menghapus node ke 2: 2 <-> 4 <->
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data double linked list merupakan struktur data dinamis yang memiliki kemampuan traversal dua arah melalui pointer next dan prev. dengan adanya dua pointer tersebut, proses pengelolaan data menjadi lebih fleksibel karena penelusuran, penambahan, dan penghapusan node dapat dilakukan dari arah depan maupun belakang.

Pada praktikum ini berhasil diimplementasikan beberapa operasi dasar double linked list menggunakan bahasa pemrograman java, yaitu insertion, traversal, dan deletion. Operasi insertion digunakan untuk menambahkan node di awal, akhir, dan posisi tertentu. Operasi traversal digunakan untuk menelusuri data secara maju maupun mundur, sedangkan operasi deletion digunakan untuk menghapus node pada bagian tertentu dalam list. Seluruh operasi berjalan dengan baik setelah pointer next dan prev diperbarui secara tepat.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, “Doubly Linked List.” [Online]. Available: <https://www.geeksforgeeks.org/doubly-linked-list/>. [Diakses: 14-Mei-2026].
- [2] Programiz, “Doubly Linked List Data Structure.” [Online]. Available: <https://www.programiz.com/dsa/doubly-linked-list>. [Diakses: 14-Mei-2026].
- [3] Programiz, “Linked List Data Structure.” [Online]. Available: <https://www.programiz.com/dsa/linked-list>. [Diakses: 14-Mei-2026].

TUGAS PRAKTIKUM STRUKTUR DATA

4.1 Soal Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

4.2 Kode program

4.2.1 Class Lagu_2511532024

```
1. public class Lagu_2511532024 {
2.     // kelas lagu
3.     String judul_2024;
4.     String penyanyi_2024;
5.     Lagu_2511532024 next_2024;
6.     Lagu_2511532024 prev_2024;
7.
8.     // konstruktor
9.     public Lagu_2511532024(String judul_2024, String
        penyanyi_2024) {
10.         this.judul_2024 = judul_2024;
11.         this.penyanyi_2024 = penyanyi_2024;
12.         this.next_2024 = null;
13.         this.prev_2024 = null;
14.     }
15. }
```

4.2.2 Class Musik_251153224

```
1. public class Musik_251153224 {
2.     Lagu_2511532024 head_2024;
3.     Lagu_2511532024 tail_2024;
4.
5.     public void tambahLagu_2024(String judul_2024,
        String penyanyi_2024) {
6.         Lagu_2511532024 laguBaru_2024 = new
            Lagu_2511532024(judul_2024, penyanyi_2024);
7.
8.         // jika playlist kosong
9.         if (head_2024 == null) {
10.             head_2024 = laguBaru_2024;
11.             tail_2024 = laguBaru_2024;
12.         } else {
13.             tail_2024.next_2024 = laguBaru_2024;
14.             laguBaru_2024.prev_2024 = tail_2024;
15.             tail_2024 = laguBaru_2024;
16.         }
17.     }
18. }
```

```

17.         System.out.println("Lagu berhasil
           ditambahkan!");
18.     }
19.
20.     // menghapus lagu pertama
21.     public void hapusLaguAwal_2024() {
22.         if (head_2024 == null) {
23.             System.out.println("Playlist kosong!");
24.             return;
25.         }
26.
27.         System.out.println("Lagu " +
           head_2024.judul_2024 + " berhasil dihapus.");
28.
29.         // jika hanya ada satu lagu
30.         if (head_2024 == tail_2024) {
31.             head_2024 = null;
32.             tail_2024 = null;
33.         } else {
34.             head_2024 = head_2024.next_2024;
35.             head_2024.prev_2024 = null;
36.         }
37.     }
38.
39.     // menampilkan playlist dari awal ke akhir
40.     public void tampilMaju_2024() {
41.         if (head_2024 == null) {
42.             System.out.println("Playlist kosong!");
43.             return;
44.         }
45.
46.         Lagu_2511532024 current_2024 = head_2024;
47.
48.         System.out.println("\n=== Playlist Maju ===");
49.         while (current_2024 != null) {
50.             System.out.println("Judul      : " +
           current_2024.judul_2024);
51.             System.out.println("Penyanyi  : " +
           current_2024.penyanyi_2024);
52.             System.out.println();
53.             current_2024 = current_2024.next_2024;
54.         }
55.     }
56.
57.     // menampilkan playlist dari akhir ke awal
58.     public void tampilMundur_2024() {
59.         if (tail_2024 == null) {
60.             System.out.println("Playlist kosong!");
61.             return;
62.         }
63.
64.         Lagu_2511532024 current_2024 = tail_2024;
65.
66.         System.out.println("\n=== Playlist Mundur
           ===");
67.         while (current_2024 != null) {

```



```

68.         System.out.println("Judul      : " +
current_2024.judul_2024);
69.         System.out.println("Penyanyi : " +
current_2024.penyanyi_2024);
70.         System.out.println();
71.         current_2024 = current_2024.prev_2024;
72.     }
73. }
74.
75. // mencari lagu berdasarkan judul
76. public void cariLagu_2024(String judulCari_2024) {
77.     if (head_2024 == null) {
78.         System.out.println("Playlist kosong!");
79.         return;
80.     }
81.     Lagu_2511532024 current_2024 = head_2024;
82.     boolean ditemukan_2024 = false;
83.
84.     while (current_2024 != null) {
85.         if
(current_2024.judul_2024.equalsIgnoreCase(judulCari_
2024)) {
86.             System.out.println("\nLagu
ditemukan!");
87.             System.out.println("Judul      : "
+ current_2024.judul_2024);
88.             System.out.println("Penyanyi : "
+ current_2024.penyanyi_2024);
89.             ditemukan_2024 = true;
90.             break;
91.         }
92.         current_2024 = current_2024.next_2024;
93.     }
94.
95.     if (!ditemukan_2024) {
96.         System.out.println("Lagu tidak
ditemukan!");
97.     }
98. }
99.
100.     public static void main(String[] args)
{
101.         Scanner sc_2024 = new
Scanner(System.in);
102.         Musik_2511532024 playlist_2024 =
new Musik_2511532024();
103.         int pilihan_2024;
104.
105.         do {
106.             System.out.println("\n===
Playlist Musik NIM: 2511532024 ===");
107.             System.out.println("1.
Tambah Lagu");
108.             System.out.println("2.
Hapus Lagu Pertama");
109.             System.out.println("3.
Lihat Playlist (Maju)");

```

```

110.         System.out.println("4.
        Lihat Playlist (Mundur)");
111.         System.out.println("5.
        Cari Lagu");
112.         System.out.println("6.
        Keluar");
113.         System.out.print("Pilihan:
        ");
114.         pilihan_2024 =
        sc_2024.nextInt();
115.         sc_2024.nextLine();
116.
117.         switch (pilihan_2024) {
118.             case 1:
119.
120.                 System.out.print("Judul: ");
        String judul_2024 =
        sc_2024.nextLine();
121.
122.                 System.out.print("Penyanyi: ");
        String
        penyanyi_2024 = sc_2024.nextLine();
123.
124.                 playlist_2024.tambahLagu_2024(judul_2024,
        penyanyi_2024);
125.
126.                 break;
127.
128.             case 2:
129.
130.                 playlist_2024.hapusLaguAwal_2024();
        break;
131.
132.             case 3:
133.
134.                 playlist_2024.tampilMaju_2024();
        break;
135.
136.             case 4:
137.
138.                 playlist_2024.tampilMundur_2024();
        break;
139.
140.             case 5:
141.
142.                 System.out.print("Masukkan judul lagu: ");
        String cari_2024 =
        sc_2024.nextLine();
143.
144.                 playlist_2024.cariLagu_2024(cari_2024);
        break;
145.
146.             case 6:
147.
148.                 System.out.println("Program selesai.");
        break;

```

```

149.
150.                                     default:
151.
152.         System.out.println("Pilihan tidak valid!");
153.     }
154.     } while (pilihan_2024 != 6);
155.         sc_2024.close();
156.     }
157. }

```

4.3 Penjelasan

4.3.1 Class Lagu_2511532024

1. Deklarasi Kelas

```
public class Lagu_2511532024
```

Syntax tersebut digunakan untuk mendeklarasikan kelas bernama Lagu_2511532024. Kelas ini berfungsi sebagai representasi node pada Double Linked List yang menyimpan data lagu.

2. Deklarasi Atribut

```
String judul_2024;
String penyanyi_2024;
Lagu_2511532024 next_2024;
Lagu_2511532024 prev_2024;
```

Syntax tersebut digunakan untuk mendeklarasikan atribut pada kelas lagu. Variabel judul_2024 digunakan untuk menyimpan judul lagu, sedangkan penyanyi_2024 digunakan untuk menyimpan nama penyanyi. Pointer next_2024 digunakan untuk menghubungkan node ke lagu berikutnya, sedangkan pointer prev_2024 digunakan untuk menghubungkan node ke lagu sebelumnya sehingga playlist dapat ditelusuri dua arah.

3. Konstruktor Lagu_2511532024

```
public Lagu_2511532024(String judul_2024, String  
penyanyi_2024)
```

Konstruktor Lagu_2511532024() digunakan untuk membuat objek lagu baru dengan menerima data judul dan penyanyi melalui parameter.

```
this.judul_2024 = judul_2024;  
this.penyanyi_2024 = penyanyi_2024;  
this.next_2024 = null;  
this.prev_2024 = null;
```

Syntax tersebut digunakan untuk mengisi atribut objek lagu dengan nilai yang diterima dari parameter konstruktor. Variabel judul_2024 dan penyanyi_2024 diisi sesuai data input, sedangkan pointer next_2024 dan prev_2024 diatur bernilai null karena node yang baru dibuat belum terhubung dengan node lain pada playlist

4.3.2 Class Musik_2511532024

1. Deklarasi Atribut

```
Lagu_2511532024 head_2024;  
Lagu_2511532024 tail_2024;
```

Syntax tersebut digunakan untuk mendeklarasikan pointer head_2024 dan tail_2024. Pointer head_2024 menunjuk node pertama pada playlist, sedangkan tail_2024 menunjuk node terakhir pada playlist.

2. Method tambahLagu_2024()

```
public void tambahLagu_2024(String judul_2024, String  
penyanyi_2024)
```

Method tambahLagu_2024() digunakan untuk menambahkan lagu baru ke bagian akhir playlist.

```
Lagu_2511532024 laguBaru_2024 = new
```

```
Lagu_2511532024(judul_2024, penyanyi_2024);
```

Syntax tersebut digunakan untuk membuat node lagu baru berdasarkan judul dan penyanyi yang diinputkan pengguna.

```
if (head_2024 == null) {  
    head_2024 = laguBaru_2024;  
    tail_2024 = laguBaru_2024;  
} else {  
    tail_2024.next_2024 = laguBaru_2024;  
    laguBaru_2024.prev_2024 = tail_2024;  
    tail_2024 = laguBaru_2024;  
}
```

Syntax tersebut digunakan untuk memeriksa apakah playlist masih kosong atau tidak. Jika playlist kosong, lagu baru akan dijadikan head_2024 dan tail_2024. Jika playlist sudah berisi lagu, maka node baru akan ditambahkan di akhir playlist dengan menghubungkan pointer next_2024 dan prev_2024.

```
System.out.println("Lagu berhasil ditambahkan!");
```

Syntax tersebut digunakan untuk menampilkan pesan bahwa lagu berhasil ditambahkan ke playlist.

3. Method hapusLaguAwal_2024()

```
public void hapusLaguAwal_2024()
```

Method hapusLaguAwal_2024() digunakan untuk menghapus lagu pertama pada playlist.

```
if (head_2024 == null) {  
    System.out.println("Playlist kosong!");  
    return;  
}
```

Syntax tersebut digunakan untuk memeriksa apakah playlist kosong. Jika kosong, program akan menampilkan pesan dan proses dihentikan menggunakan return.

```
System.out.println("Lagu " + head_2024.judul_2024 + "
berhasil dihapus.");
```

Syntax tersebut digunakan untuk menampilkan informasi lagu yang berhasil dihapus dari playlist.

```
if (head_2024 == tail_2024) {
    head_2024 = null;
    tail_2024 = null;
} else {
    head_2024 = head_2024.next_2024;
    head_2024.prev_2024 = null;
}
```

Syntax tersebut digunakan untuk menangani dua kondisi penghapusan. Jika playlist hanya memiliki satu lagu, maka head_2024 dan tail_2024 diubah menjadi null. Jika playlist memiliki lebih dari satu lagu, maka posisi head_2024 dipindahkan ke node berikutnya dan pointer prev_2024 pada head baru dihapus.

4. Method tampilMaju_2024()

```
public void tampilMaju_2024()
```

Method tampilMaju_2024() digunakan untuk menampilkan playlist dari awal ke akhir.

```
if (head_2024 == null) {
    System.out.println("Playlist kosong!");
    return;
}
Lagu_2511532024 current_2024 = head_2024;
```

Syntax tersebut digunakan untuk memeriksa apakah playlist kosong dan membuat pointer bantu bernama current_2024 yang digunakan untuk traversal mulai dari head.

```
while (current_2024 != null) {
    System.out.println("Judul          : " +
```

```

current_2024.judul_2024);
        System.out.println("Penyanyi      :      "      +
current_2024.penyanyi_2024);
        System.out.println();
        current_2024 = current_2024.next_2024;
    }

```

Syntax tersebut digunakan untuk menelusuri playlist dari depan ke belakang menggunakan pointer next_2024 sambil menampilkan data lagu satu per satu.

5. Method tampilMundur_2024()

```

public void tampilMundur_2024()

```

Method tampilMundur_2024() digunakan untuk menampilkan playlist dari akhir ke awal.

```

if (tail_2024 == null) {
    System.out.println("Playlist kosong!");
    return;
}
Lagu_2511532024 current_2024 = tail_2024;

```

Syntax tersebut digunakan untuk memeriksa apakah playlist kosong dan membuat pointer bantu yang memulai traversal dari tail.

```

while (current_2024 != null) {
    System.out.println("Judul      :      "      +
current_2024.judul_2024);
    System.out.println("Penyanyi      :      "      +
current_2024.penyanyi_2024);
    System.out.println();
    current_2024 = current_2024.prev_2024;
}

```

Syntax tersebut digunakan untuk menampilkan playlist dari belakang ke depan menggunakan pointer prev_2024.

6. Method cariLagu_2024()

```
public void cariLagu_2024(String judulCari_2024)
```

Method cariLagu_2024() digunakan untuk mencari lagu berdasarkan judul.

```
if (head_2024 == null) {  
    System.out.println("Playlist kosong!");  
    return;  
}  
Lagu_2511532024 current_2024 = head_2024;  
boolean ditemukan_2024 = false;
```

Syntax tersebut digunakan untuk memeriksa apakah playlist kosong, membuat pointer traversal, dan membuat variabel penanda apakah lagu ditemukan atau tidak.

```
while (current_2024 != null) {  
    if  
(current_2024.judul_2024.equalsIgnoreCase(judulCari_2024)) {  
        System.out.println("\nLagu ditemukan!");  
        System.out.println("Judul      : " +  
current_2024.judul_2024);  
        System.out.println("Penyanyi   : " +  
current_2024.penyanyi_2024);  
        ditemukan_2024 = true;  
        break;  
    }  
    current_2024 = current_2024.next_2024;  
}
```

Syntax tersebut digunakan untuk melakukan pencarian lagu dengan traversal dari depan ke belakang. Method equalsIgnoreCase() digunakan agar pencarian tidak membedakan huruf besar dan kecil.


```
if (!ditemukan_2024) {  
    System.out.println("Lagu tidak ditemukan!");  
}
```

Syntax tersebut digunakan untuk menampilkan pesan apabila lagu yang dicari tidak ditemukan di dalam playlist.

7. Method main()

```
Scanner sc_2024 = new Scanner(System.in);  
Musik_2511532024    playlist_2024    =    new  
Musik_2511532024();  
int pilihan_2024;
```

Syntax tersebut digunakan untuk membuat objek Scanner, membuat objek playlist, dan mendeklarasikan variabel pilihan menu.

```
do {  
    ...  
} while (pilihan_2024 != 6);
```

Syntax tersebut digunakan untuk menjalankan menu program secara berulang sampai pengguna memilih menu keluar.

```
System.out.println("\n===    Playlist    Musik    NIM:  
2511532024 ===");  
System.out.println("1. Tambah Lagu");  
System.out.println("2. Hapus Lagu Pertama");  
System.out.println("3. Lihat Playlist (Maju)");  
System.out.println("4. Lihat Playlist (Mundur)");  
System.out.println("5. Cari Lagu");  
System.out.println("6. Keluar");
```

Syntax tersebut digunakan untuk menampilkan daftar menu program kepada pengguna.

```
pilihan_2024 = sc_2024.nextInt();  
sc_2024.nextLine();
```

Syntax tersebut digunakan untuk membaca input pilihan menu dari pengguna dan membersihkan karakter newline yang tersisa pada buffer input.

```
switch (pilihan_2024)
```

Syntax tersebut digunakan untuk menjalankan fitur program sesuai pilihan pengguna.

```
case 1:
```

```
    System.out.print("Judul: ");  
    String judul_2024 = sc_2024.nextLine();  
    System.out.print("Penyanyi: ");  
    String penyanyi_2024 = sc_2024.nextLine();  
    playlist_2024.tambahLagu_2024(judul_2024,  
    penyanyi_2024);  
    break;
```

Syntax tersebut digunakan untuk menerima input judul dan penyanyi, kemudian menambahkan lagu ke playlist.

```
case 2:
```

```
    playlist_2024.hapusLaguAwal_2024();  
    break;
```

```
case 3:
```

```
    playlist_2024.tampilMaju_2024();  
    break;
```

```
case 4:
```

```
    playlist_2024.tampilMundur_2024();  
    break;
```

Syntax tersebut digunakan untuk menjalankan fitur menghapus lagu pertama, menampilkan playlist maju, dan menampilkan playlist mundur.

```
case 5:
```

```
    System.out.print("Masukkan judul lagu: ");
```

```
String cari_2024 = sc_2024.nextLine();  
playlist_2024.cariLagu_2024(cari_2024);  
break;
```

Syntax tersebut digunakan untuk menerima input judul lagu yang ingin dicari dan menjalankan proses pencarian lagu.

case 6:

```
System.out.println("Program selesai.");  
break;
```

default:

```
System.out.println("Pilihan tidak valid!");
```

Syntax tersebut digunakan untuk keluar dari program atau menampilkan pesan apabila pilihan menu tidak tersedia.

```
sc_2024.close();
```

Syntax tersebut digunakan untuk menutup objek Scanner setelah program selesai dijalankan.

4.4 Output Kode Program

```
=== Playlist Musik NIM: 2511532024 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 1  
Judul: who knows  
Penyanyi: daniel caesar  
Lagu berhasil ditambahkan!  
  
=== Playlist Musik NIM: 2511532024 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 1  
Judul: backburner  
Penyanyi: niki  
Lagu berhasil ditambahkan!  
  
=== Playlist Musik NIM: 2511532024 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 1  
Judul: risk it all  
Penyanyi: bruno mars  
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist Maju ===
Judul   : who knows
Penyanyi : daniel caesar

Judul   : backburner
Penyanyi : niki

Judul   : risk it all
Penyanyi : bruno mars

=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== Playlist Mundur ===
Judul   : risk it all
Penyanyi : bruno mars

Judul   : backburner
Penyanyi : niki

Judul   : who knows
Penyanyi : daniel caesar

=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: backburner

Lagu ditemukan!
Judul   : backburner
Penyanyi : niki

=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu who knows berhasil dihapus.

=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist Maju ===
Judul   : backburner
Penyanyi : niki

Judul   : risk it all
Penyanyi : bruno mars

=== Playlist Musik NIM: 2511532024 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai.
```